DARI 검증 아키텍처 v0.3

Graph 중심 검증에서 Claim Ledger 중심의 provenance-aware semantic diff로

사용자가 확인한 claim과 원문 근거가 사실 추출, 초안, 수정, 번역 과정에서 보존되는지를 추적한다. Graph는 관계 오류에서 추가 가치를 증명할 때만 사용한다.

판단	현재 평가	의미
Impact 잠재력	높음	행위자·부정·확실성·출처 변화는 고위험 문서의 의미를 바꿀 수 있음
현재 기술 성숙도	낮음	자동 extractor와 aligner는 아직 성능 증거가 없음
기술적 해자	조건부	graph 자체보다 provenance, correction data, calibration에 있음
첫 구현 범위	V2	confirmed facts에서 한국어 draft로 의미가 보존되는지 검사
핵심 중단 기준	명시	strongest baseline을 못 이기면 graph를 축소 또는 제거

작성 근거

DARI 기획서와 검증시스템 자료조사, 현재 `capacitor-web`의 fact/evidence/revision 구조, 기존 verification 실험 코드와 문서를 함께 검토했다. 연구 문헌은 의미 역할, graph 표현, 오류 span, challenge set, uncertainty calibration의 아이디어를 선택적으로 반영했다.

1. 왜 기존 설계를 다시 고쳤는가

v0.2는 contract, evidence, 재현성, baseline 비교를 치밀하게 설계했다. 그러나 source와 candidate를 모두 graph로 바꾸는 구조가 DARI의 실제 강점과 충분히 연결되지 않았고, runtime 모듈과 연구용 비교기가 한 그림에 함께 있어 제품의 중심 가설이 흐려졌다.

관찰	문제	v0.3 결정
Reference까지 매번 LLM graph 추출	추출 오류가 양쪽에 들어가고 같은 모델의 상관 오류가 생김	사용자 confirmed facts로 Claim Ledger를 결정론적으로 구성
Graph가 모든 오류를 처리	단순 값 오류에도 비싼 구조를 사용	Graph를 role·scope·time 관계 오류에만 사용
V1/V2/V3 동시 구현	서로 다른 병목이 섞여 원인을 알기 어려움	V2 same-language draft 검증부터 시작
QE·judge·graph runtime 병렬화	복잡도와 비용이 늘고 기여 분리가 어려움	baseline은 offline research plane으로 분리
Macro-F1 중심	실제 경고 피로와 검수 개선을 설명하지 못함	fixed burden recall, false alerts, residual error, review time 사용

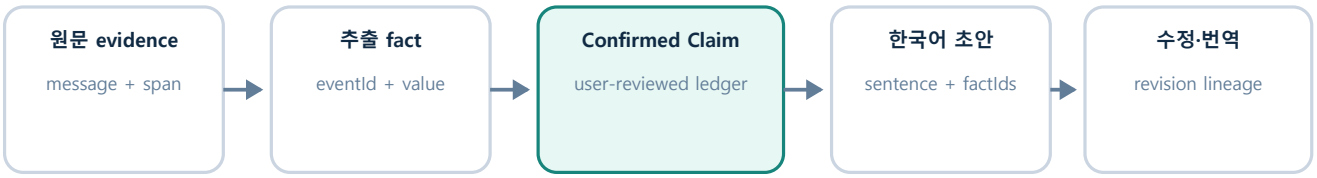
현재 demo가 실제로 증명한 것

- Hand-authored graph가 주어졌을 때 일부 role, polarity, certainty, attribution, temporal diff가 동작한다.
- Evidence span과 finding을 strict report contract로 연결할 수 있다.
- Graph trace와 module output을 viewer에서 확인할 수 있다.

아직 증명하지 못한 것

- 자유 문단에서 claim과 graph를 자동으로 정확히 추출하는 능력
- 실제 의역과 문장 분할·병합에서 entity/event를 정렬하는 능력
- Direct judge, flat claim + NLI, xCOMET보다 중요한 오류를 더 잘 찾는다는 주장
- Finding이 사람의 검수 정확도와 시간을 실제로 개선한다는 주장

2. 중심축: Semantic chain of custody



하나의 알고리즘이 아니라 claim_id · evidence · revision으로 이어지는 semantic chain of custody

세 단계는 같은 graph algorithm이 아니다. 각 단계는 서로 다른 입력과 gold를 가진다. 대신 claim ID, evidence, revision을 이어 받아 최종 오류가 어디에서 들어왔는지 역추적한다.

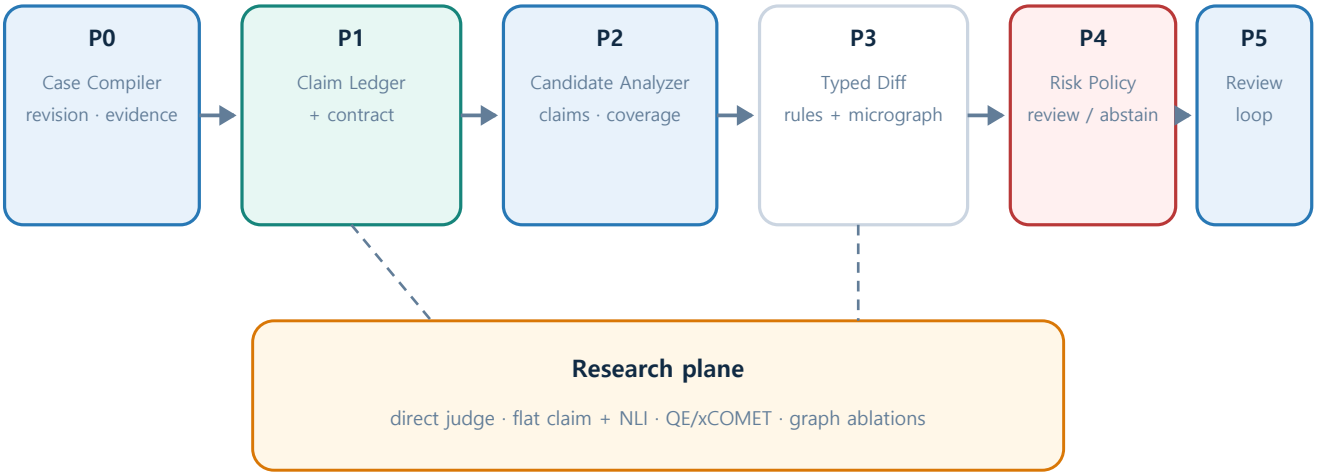
경계	Reference	Candidate	핵심 질문
V1 extraction	사용자 원문 evidence	추출 fact	원문이 fact로 정확히 구조화됐는가
V2 drafting	confirmed Claim Ledger	한국어 draft	확인 의미가 초안에서 변형됐는가
V3 translation	확정 draft claims	다국어 translation	언어 변환 중 의미가 변했는가

왜 V2부터 시작하는가

- 현재 제품에는 confirmed facts, sourceQuotes, eventId, certainty, knowledge, supersedes, factIds가 이미 있다.
- 같은 언어의 confirmed facts와 한국어 draft를 비교하면 cross-lingual extraction과 alignment 실패를 먼저 섞지 않는다.
- 현재 `factIds` 검사는 연결 존재 여부는 보지만, 문장이 fact 의미를 실제로 보존했는지는 보장하지 않는다.
- V2에서 검증한 확정 draft claims를 V3 reference로 재사용하면 비용과 reference-side 오류를 줄일 수 있다.

제품의 첫 질문은 '번역 점수가 좋은가?'가 아니라 '사용자가 확인한 claim이 초안에서 그대로 살아 있는가?'이다.

3. 전체 아키텍처



runtime은 선택된 방법 하나만 실행하고, baseline 비교는 offline에서 분리한다.

모듈	Input	Output	핵심 책임
P0 Case Compiler	artifact, revision, digest, stage	VerificationCaseV2	stale artifact와 잘못된 연결을 모델 호출 전에 제거
P1 Claim Ledger	confirmed facts, sourceQuotes, corrections	ClaimLedger + Contract	reference와 이번 문서의 필수 의미 범위 고정
P2 Candidate Analyzer	candidate text, factIds, language	candidate claims + coverage	candidate만 보고 의미와 evidence 추출
P3 Typed Diff	contract + 양쪽 claims	candidate findings	단순 invariant와 관계 micrograph를 결합
P4 Risk Policy	findings, confidence, coverage	review / pass / abstain	언어·오류 유형별 calibrated routing
P5 Review Loop	finding + 양쪽 evidence	accepted/rejected/corrected	사람 판단과 correction revision 연결

Research plane을 분리하는 이유

Direct judge, flat claim + NLI, QE/xCOMET, symmetric full-graph는 제품에서 동시에 섞지 않는다. 같은 frozen data와 비용 조건에서 독립 실행하고, 실제 우위를 보인 방법만 runtime 후보로 채택한다.

4. 핵심 artifact

Artifact	포함 내용	왜 필요한가
Evidence Ledger	message, language, revision, exact spans	사용자의 실제 표현으로 모든 판단을 되돌릴 수 있음
Canonical Claim Ledger	event, roles, value, certainty, knowledge, correction	사용자 확인 의미를 재추출 없이 reference로 재사용
Semantic Contract	required, optional, forbidden, allowed transformations	정상 요약이나 문항별 선택을 omission으로 오판하지 않음
Candidate Semantic Artifact	candidate claims, relations, evidence, coverage	초안 또는 번역문에서 실제 관찰된 의미를 분리
Finding	claim ID, type, risk, 양쪽 evidence, detector state	사용자가 판단하고 수정할 수 있는 최소 단위

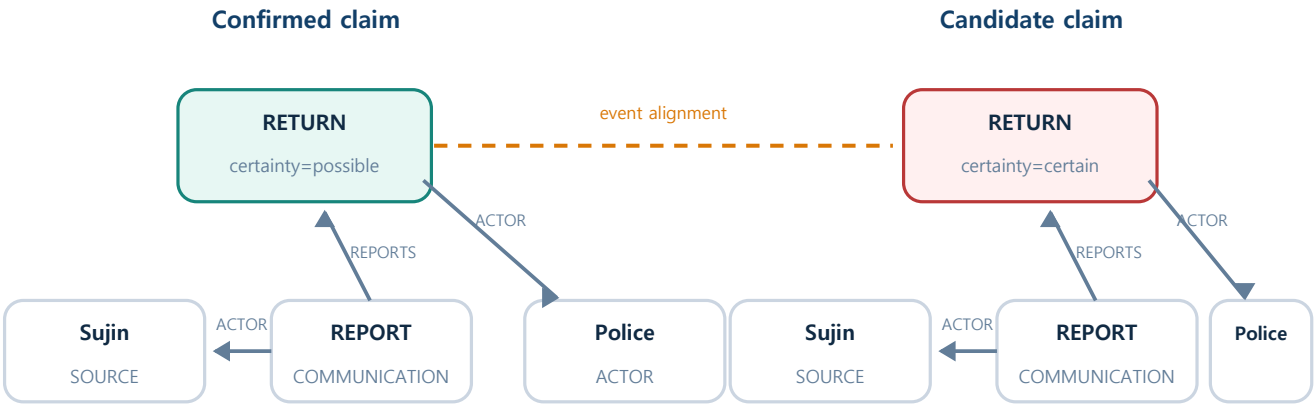
Semantic Contract가 중요한 이유

문항 하나의 답변은 Case 전체를 복사하는 문서가 아니다. 어떤 claim은 반드시 들어가야 하고, 어떤 claim은 선택적이며, 정정으로 폐기된 claim은 들어가면 안 된다. 이 stage-specific contract가 없으면 정밀한 graph도 잘못된 목표를 검사하게 된다.

Contract field	의미	예시
required_claim_ids	이번 section이 반드시 보존할 의미	특정 사건의 행위자, 날짜, 전언 여부
optional_claim_ids	빠져도 오류가 아닌 배경	다른 문항에 이미 설명한 주변 맥락
forbidden_claim_ids	정정으로 폐기되거나 범위를 벗어난 의미	March를 April로 정정한 뒤의 March claim
allowed_transformations	정상적인 표현 변화	호칭 변화, 문장 분할·병합, 간결한 요약

Claim Ledger는 세계의 진실이 아니라 사용자가 확인한 현재 기준이다. 이 한계를 명시해야 시스템을 법률 판단기로 오해하지 않는다.

5. Graph를 어디에 쓰고 어디에서 빼는가



event 대응은 유지하고 qualifier 차이만 certainty_shift로 보고한다.

검사	우선 경로	Graph가 필요한 조건
Required claim 누락	Semantic Contract coverage	event merge/split 때문에 claim 대응이 모호할 때
숫자·날짜·이름	normalized value 비교	값이 여러 event에 속해 대응이 모호할 때
명시적 부정	deterministic lexical + claim polarity	부정 scope가 여러 claim 중 하나에만 걸릴 때
행위자·대상	Relational micrograph	항상 관계 구조가 필요
정보 출처	REPORT event + REPORTS relation	화자와 내용 event의 연결을 비교
확실성	Scoped qualifier	어떤 event 또는 relation에 적용되는지 비교
시간 순서	BEFORE/AFTER/FREQUENCY claims	문장 순서와 사건 순서가 다른 경우

MVP type

- omission, actor_role_change, negation_flip, certainty_shift
- source_attribution_change, identity_value_change, temporal_change

Causality와 harm strength는 annotation agreement와 실제 빈도를 확인한 뒤 추가한다. 특히 피해 강도는 법률적 판단과 혼동될 수 있어 초기 자동 판정에서 제외한다.

6. 논문 아이디어를 어떻게 재구성했는가

연구 계열	가져온 아이디어	DARI에서의 재구성
PropBank / MEANT	predicate와 semantic role 비교	전체 SRL score 대신 고위험 role 차이를 typed finding으로 사용
AMR / Smatch	node-edge meaning representation	full AMR 대신 event 주변의 작은 relation subgraph
S2MATCH / FGW	soft semantic alignment	embedding은 후보 검색에만 사용하고 최종 판정은 rule로 제한
COMETKiwi / xCOMET	QE, fine-grained error span	제품 core가 아니라 강한 외부 baseline
QuestEval	QA 기반 정보 보존 확인	전체 문서 QA 대신 모호한 claim만 선택적으로 재확인
ACES / SPAN-ACES	현상별 challenge set과 source-sensitive 평가	DARI 관계 오류 중심 frozen test와 span evidence
Conformal MT evaluation	그룹별 uncertainty calibration	언어·길이·오류 유형별 review burden 보정

중요한 해석

ACES 계열 연구는 평균 상관이 높은 metric도 특정 의미 현상에서 실패할 수 있고, source를 약하게 보는 metric과 LLM evaluator가 불안정할 수 있음을 보여준다. 따라서 DARI는 전체 점수보다 error label, source evidence, 현상별 challenge set을 중심으로 평가한다.

Conformal MT evaluation의 문제의식에 따라 model confidence를 그대로 사용자에게 노출하지 않는다. Calibration set에서 risk-coverage를 측정하고, 불확실한 사례는 보류한다.

주요 참고문헌

- Lo and Wu (2013), MEANT: https://aclanthology.org/W13-2254/
- Banarescu et al. (2013), AMR: https://aclanthology.org/W13-2322/
- Opitz et al. (2020), S2MATCH: https://aclanthology.org/2020.tacl-1.34/
- Guerreiro et al. (2024), xCOMET: https://aclanthology.org/2024.tacl-1.54/
- Moghe et al. (2025), ACES/SPAN-ACES: https://aclanthology.org/2025.cl-1.4/
- Zerva and Martins (2024), Conformal MT Evaluation: https://aclanthology.org/2024.tacl-1.80/

7. 객관적인 평가 설계

단계	규모	결정 질문
Representation audit	자연 문단 60-100	실제 중요 오류의 95%를 claim/micrograph로 표현 가능한가
Oracle comparator	gold claim + alignment	Typed diff 자체가 high-risk recall 95%, faithful FPR 5%를 달성하는가
Feasibility pilot	180 pairs	Extractor, aligner, annotation 중 병목이 무엇인가
Frozen challenge	약 600 pairs	현상별 성능과 baseline 우위가 재현되는가
Ecological test	약 400 pairs	실제 prevalence에서 precision과 warning burden이 유지되는가
User crossover	사전 power analysis	잔존 오류 또는 검수 시간이 개선되는가

가장 중요한 baseline

Baseline	검증하는 것
Deterministic guards	Graph가 싼 규칙보다 못하지 않은가
Flat claim + bidirectional NLI	Relation edge가 실제 추가 가치를 만드는가
Cost-matched direct judge	같은 기반 모델과 비용에서 구조화가 유리한가
COMETKiwi/xCOMET	Translation stage에서 강한 일반 QE보다 나은가

Oracle decomposition

- A0 gold graph + gold alignment: schema와 comparator의 상한
- A1 predicted claims + gold alignment: extractor 손실
- A2 predicted claims + predicted alignment: aligner 손실
- A3 A2 + verifier: 오탐 감소 효과
- A4 flat claims: graph edge 자체의 가치
- A7 symmetric full graph: Claim Ledger reference 재사용의 가치

8. 제품 impact 지표와 통과 기준

Macro-F1은 진단용으로 남기되, 제품 판단은 사용자가 감당할 수 있는 검토 부담에서 중요한 오류를 얼마나 줄이는지로 한다.

지표	정의	왜 중요한가
High-risk recall at fixed burden	faithful FPR 또는 문서당 경고 수를 고정한 recall	경고를 무한히 늘려 얻은 recall을 배제
Residual high-risk error	검토 후에도 남은 중요 오류	사용자의 최종 결과와 직접 연결
False alerts per faithful paragraph	정상 문단 하나당 불필요한 finding	긴 문서의 경고 피로를 드러냄
Review time	같은 최종 정확도에 필요한 시간	실제 workflow 효율을 측정
Evidence sufficiency	표시된 근거만으로 정확히 판단 가능한 비율	설명 가능성이 행동으로 이어지는지 측정

Gate	Go	실패 시 결정
Comparator	gold 조건 recall 95%+, faithful FPR 5%-	schema/diff 재설계
자동 분석	oracle recall의 85%+ 보존	extractor 데이터와 모델에 집중
Graph 가치	strongest baseline보다 recall +8%p, CI > 0	conditional module로 축소 또는 제거
Burden	같은 recall에서 false alert 30%+ 감소	risk policy와 evidence UI 재설계
User impact	잔존 오류 또는 검수 시간 20%+ 감소	연구 도구로만 유지
Generalization	두 번째 언어·모델에서 이득 80%+ 유지	지원 범위 축소

Frozen test를 본 뒤 기준을 낮추면 검증이 아니다. Pilot은 기준을 바꾸는 단계가 아니라 분산과 표본 수를 정하는 단계다.

9. 기술적 해자는 어디에 있는가

구분	요소	평가
복제 쉬움	graph schema, prompt, embedding, matching, viewer	논문과 범용 모델로 재현 가능
축적 자산	claim/evidence/correction revision lineage	제품 workflow 안에서만 자연스럽게 쌓임
데이터 해자	사용자가 수락·거절·수정한 typed finding	DARI 문맥의 hard negative와 calibration data
평가 해자	이중언어 gold set, codebook, natural error benchmark	경쟁 방식의 약점을 재현 가능하게 측정
운영 해자	언어별 risk policy와 privacy-aware execution	실제 사용자 환경에 맞춘 신뢰도 관리

해자 주장도 실험해야 한다

DARI gold 사례를 0, 50, 150, 300개로 늘리며 micrograph, flat claim, direct judge의 learning curve를 비교한다. DARI 구조를 쓰는 방법의 개선 기울기가 더 크고, 기반 모델과 언어쌍을 바꿔도 이득이 유지될 때만 데이터 해자로 부를 수 있다.

Go	Pivot	Kill
관계 오류에서 명확한 recall 우위 evidence가 정확하고 검수 부담 감소 사용자 결과 20% 이상 개선	Claim Ledger 가치는 있으나 graph 이득이 작음 deterministic + NLI/direct judge로 단순화 graph는 일부 고위험 관계에만 사용	자연 데이터에서 baseline과 차이 없음 경고 수와 검수 시간이 증가 언어별 abstention이 과도함

Graph는 기술적 해자가 아니라 해자를 만드는 데이터를 구조화하는 내부 표현이다.

10. 구현 로드맵

순서	구현	완료 기준
1	VerificationCaseV2, ClaimLedger, SemanticContract, multi-span evidence	제품 artifact를 손실 없이 표현
2	IntakeFact -> ClaimLedger deterministic projector	confirmed/correction/evidence lineage 재현
3	V2 invariant guards	coverage, old value, certainty, knowledge 검사
4	한국어 candidate claim extractor	gold subset에서 claim extraction 측정
5	Typed diff oracle test	Comparator gate 통과
6	Flat claim, direct judge baseline runner	같은 data와 비용으로 비교
7	Relational micrograph aligner	관계 오류 subset에서 추가 가치 확인
8	Selective verifier	반복 오탐을 실제로 줄일 때만 유지
9	V3 cross-lingual extension	두 번째 언어쌍에서 재현
10	Review adapter and user study	잔존 오류 또는 검수 시간 개선

현재 코드 재사용

- `contracts.py`: strict validation, evidence, report state
- `pipeline.py`: failure isolation, fingerprint, indeterminate handling
- `fixtures.py`와 `batch.py`: dataset digest와 독립 실행
- `graph_demo.py`: typed diff oracle prototype
- `graph_viewer/`: candidate claim과 micrograph trace 디버깅
- `IntakeFact`, draft `factIds`, revision state: Claim Ledger와 freshness의 제품 기반

의도적으로 뒤로 미루는 것

- 모든 stage의 symmetric full graph extraction
- 초기 10개 이상 오류 type 지원
- harm strength의 자동 판정
- 복잡한 FGW 또는 일반 1:N graph matching
- 성능 증거 전 ensemble과 fusion
- 알고리즘 가치 검증 전 on-device 최적화

11. 비판적 최종 평가

관점	점수	평가
문제 중요도	9/10	의미 drift가 고위험 문서의 사실 관계와 신뢰성에 영향을 줄 수 있음
제품 적합성	8/10	DARI가 이미 수집하는 evidence와 correction 구조를 직접 활용
현재 성숙도	3/10	자동 extractor와 aligner가 아직 oracle fixture 수준
기술 차별성	5/10	workflow는 차별적이지만 개별 알고리즘은 공개 기술
해자 잠재력	7/10	실사용 correction data와 calibration이 축적될 경우 강해짐
구현 가능성	7/10	V2와 7개 type으로 범위를 줄여 현실성이 높아짐
실패 위험	7/10	저자원 parsing, 오탐 피로, reference 오류, 사용자 효과 불확실성
현재 impact 증거	2/10	실제 사용자와 자연 데이터 결과가 아직 없음

가장 큰 위험 네 가지

- Claim Ledger가 틀리면 시스템은 잘못된 의미를 일관되게 보존할 수 있다.
- 저자원 언어의 coreference와 생략 주어에서 candidate extractor가 무너질 수 있다.
- Ontology가 경직되면 정상적인 요약과 의역을 오류로 잡을 수 있다.
- Offline F1이 좋아도 finding 수가 많으면 사용자 검수 결과는 나빠질 수 있다.

v0.3의 가장 강한 아이디어는 graph 비교가 아니라 provenance-aware staged verification이다. Graph는 관계 오류에서 실질적 추가 성능을 보일 때만 남긴다.

최종 의사결정

지금 단계에서 이 설계는 진행할 가치가 있다. 다만 'graph 기반이라서 우수하다'고 주장할 단계는 아니다. 먼저 V2 Claim Ledger 기반 검증을 구현하고, flat claim + NLI 및 cost-matched direct judge와 비교해야 한다. 관계 오류에서 명확한 추가 이득과 낮은 검수 부담이 동시에 확인될 때 micrograph를 제품 core로 승격한다. 그렇지 않으면 Claim Ledger 중심의 더 단순한 시스템으로 전환하는 것이 기술적으로 더 정직하고 제품적으로도 낫다.

설계의 핵심 문장

사용자가 확인한 의미를 claim과 evidence로 고정하고, 각 변환 단계에서 semantic checksum을 계산하며, 사람의 수정 결과로 시스템을 개선한다.

이 보고서는 오류의 실제 성능을 증명한 결과 보고서가 아니다. 다음 구현과 실험이 무엇을 증명해야 하는지, 실패하면 어떤 설계를 버릴지까지 고정한 architecture decision report다.